Лабораторная работа № 5
Кэширование данных с использованием Redis и исследование его влияния на производительность веб-приложения

Цель работы

- изучить влияние кэширования на производительность веб-приложения;
- познакомиться с использованием Redis в качестве внешнего хранилища кэша;
- реализовать стратегию Cache-Aside;
- исследовать различия между Cache Hit и Cache Miss;
- измерить влияние кэширования на RPS, latency, p95 и p99;
- исследовать влияние TTL на эффективность кэширования;
- оценить изменение нагрузки на основной источник данных;
- использовать k6, Prometheus и Grafana для сравнительного анализа;
- определить условия, при которых кэширование повышает производительность системы.

Кэширование в высоконагруженных приложениях

Кэширование (Caching) – механизм временного хранения результатов вычислений или ранее полученных данных для ускорения повторного доступа к ним.

В высоконагруженных приложениях значительная часть запросов может обращаться к одним и тем же данным.

Кэширование позволяет сохранить результат обращения к источнику данных и использовать его при последующих запросах.



При повторном запросе данные могут быть получены непосредственно из Redis без обращения к основному источнику.

Redis

`Redis` – система хранения данных в памяти, поддерживающая различные структуры данных и операции над ними.

В работе Redis используется как распределённый кэш, доступный приложению по сети. Использование внешнего кэша позволяет нескольким экземплярам приложения обращаться к общим кэшированным данным. Это важно при горизонтальном масштабировании.

Cache Hit и Cache Miss

`Cache Hit` – ситуация, когда запрашиваемые данные найдены в кэше.
`Cache Miss` – ситуация, когда необходимые данные отсутствуют в кэше и приложение обращается к основному источнику.

Доля попаданий в кэш (Hit Ratio) рассчитывается по формуле:

$$\text{HitRatio} = \frac{\text{Hits}}{\text{Hits} + \text{Misses}} 100\%$$

Чем выше доля попаданий, тем больше запросов потенциально могут быть обработаны без обращения к основному источнику.

Однако высокая доля попаданий сама по себе не гарантирует улучшения производительности. Необходимо также учитывать стоимость обращения к Redis, размер данных и характер нагрузки.

Стратегия Cache-Aside

Будем использовать стратегию Cache-Aside. При получении запроса приложение:

1. проверяет наличие данных в Redis.
2. если данные найдены, возвращает их из кэша.
3. если данные отсутствуют, обращается к основному источнику.
4. сохраняет полученный результат в Redis.
5. возвращает результат клиенту.

Преимущество стратегии – кэшируются преимущественно данные, к которым действительно обращаются пользователи.

Недостатки – первый запрос выполняется без ускорения, а при истечении срока хранения возможны повторные обращения к основному источнику.

Время жизни кэша

`TTL (Time To Live)` – время, в течение которого запись сохраняется в кэше до автоматического истечения срока действия.

После истечения TTL следующий запрос к соответствующему ключу приводит к Cache Miss. Малый TTL способствует более частому обновлению данных, но увеличивает нагрузку на источник. Большой TTL может повысить долю попаданий, однако увеличивает период потенциального устаревания данных.

Влияние кэширования на производительность

Для оценки эффективности кэширования используются следующие метрики.

Метрика	**Назначение**
RPS	Фактическое количество HTTP-запросов в секунду
p50	Медианное время ответа
p95	95-й процентиль времени ответа
p99	99-й процентиль времени ответа
Error Rate	Доля неуспешных запросов
Hit Ratio	Доля попаданий в кэш
Miss Ratio	Доля промахов кэша
Source RPS	Количество обращений к основному источнику в секунду

Эффективность кэширования можно оценивать по изменению времени ответа и максимальной устойчивой пропускной способности.

Коэффициент ускорения по времени ответа:

$$S_L = \frac{L_{without}}{L_{with}}$$

где $L_{without}$ и L_{with} – сопоставимые значения задержки без кэша и с кэшем.

Коэффициент увеличения пропускной способности:

$$S_R = \frac{RPS_{without}}{RPS_{with}}$$

Для корректного расчёта необходимо сравнивать максимальную устойчивую пропускную способность при одинаковых критериях допустимых задержек и ошибок.

Шаг 1. Проверка Kubernetes
Выполнить:
```
kubectl get nodes
kubectl get pods
kubectl get services
```
Убедиться, что кластер доступен.

Шаг 2. Развёртывание Redis
Создать файл redis.yaml:
```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: redis
spec:
  replicas: 1
  selector:
    matchLabels:
      app: redis
  template:
    metadata:
      labels:
        app: redis
    spec:
      containers:
        - name: redis
          image: redis:7-alpine
          ports:
            - containerPort: 6379
          resources:
            requests:
              cpu: 100m
              memory: 64Mi
            limits:
              cpu: 500m
              memory: 256Mi
---
apiVersion: v1
kind: Service
metadata:
  name: redis
spec:
  selector:
    app: redis
  ports:
    - port: 6379
      targetPort: 6379
```

Развернуть Redis:
```
kubectl apply -f redis.yaml
```

Проверить состояние:
```
kubectl rollout status deployment/redis
```

```
kubectl get pods -l app=redis
kubectl get service redis
```

Шаг 3. Проверка Redis

Получить имя Pod:
kubectl get pods -l app=redis

Подключиться к Redis:
kubectl exec -it deployment/redis -- redis-cli

Примечание. Приведённая конфигурация предназначена только для ***локальной учебной*** среды. Она не предусматривает постоянного хранения, аутентификации, высокой доступности и других механизмов, необходимых для эксплуатации Redis в производственной системе.

Подготовка приложения

Архитектура приложения
Для исследования используется HTTP-endpoint:
`/data/<item_id>`

Он возвращает информацию об объекте с указанным идентификатором.
В приложении предусмотрены два режима:
– cache=off – получение данных непосредственно из источника;
– cache=on – получение данных с использованием Redis.
При отсутствии данных в кэше приложение выполняет имитацию медленного обращения к источнику.
Это позволяет сравнивать одинаковые запросы в разных режимах.

Создание приложения

Будем использовать Flask-приложение – сервис для сравнения работы с кэшем (Redis) и без него, с полным набором метрик Prometheus. Кратко:

Основной эндпоинт `/data/<item_id>`

Принимает параметр `cache=on|off (по умолчанию off).`

В режиме on: сначала ищет значение в Redis по ключу product:<id>; при промахе – загружает из «медленного источника» (имитация через time.sleep(0.05)) и кэширует с TTL (CACHE_TTL, по умолчанию 30 сек).

В режиме off: всегда идёт в источник. Возвращает JSON с id, значением и режимом кэша. При недоступности Redis – 503 с ошибкой.

Метрики Prometheus (/metrics)

cache_hits_total / cache_misses_total – попадания/промахи кэша.

source_requests_total – обращения к источнику.

app_http_requests_total{mode,status} – HTTP-запросы с разбивкой по режиму и статусу.

app_http_request_duration_seconds{mode} – гистограмма времени ответа.

Прочее

/health – healthcheck (200 OK).

Redis-клиент настроен через env (REDIS_HOST, CACHE_TTL) с таймаутами по 1 сек.

Примечание: задержка источника данных составляет 50 мс. Это искусственно заданная задержка, используемая для эксперимента. В реальной системе задержка зависит от базы данных, сетевого взаимодействия и сложности запроса.

Зависимости (requirements.txt)
Flask, redis, prometheus-client, gunicorn.

Dockerfile
Базовый образ python:3.12-slim, рабочая директория /app.

Сборка образа

Манифест cache-app.yaml
Deployment: 1 реплика, образ cache-lab:1.0, порт 8000, env REDIS_HOST=redis и CACHE_TTL=30, readinessProbe на /health, ресурсы (requests 100m/128Mi, limits 1 CPU/256Mi).

Service: порт 80 → targetPort 8000.

Применение
`kubectl apply -f cache-app.yaml, проверка через rollout status и get pods.`

Проверка работы
`kubectl port-forward service/cache-app 8080:80,`
затем curl с cache=off и дважды с cache=on – первый запрос идёт в источник, повторный (пока жив TTL) – из Redis.

Задания для самостоятельного выполнения

Задание 1. Сравнение производительности
Провести нагрузочное тестирование приложения в двух режимах:
- без кэширования;
- с использованием Redis.

При одинаковой интенсивности запросов измерить RPS, p50, p95, p99 и Error Rate.
Построить сравнительный график p95.

Задание 2. Оценка пропускной способности
Постепенно увеличивать интенсивность нагрузки и определить максимальный устойчивый RPS для каждого режима.
Использовать одинаковые критерии:
- p95 меньше 500 мс;
- Error Rate меньше 1%;
- отсутствие dropped_iterations.

Вычислить коэффициент увеличения пропускной способности.

Задание 3. Исследование эффективности кэша
Провести испытания при 10, 100 и 1000 различных объектах.
Измерить Hit Ratio, p95 и Source RPS.
Объяснить полученные зависимости.

Задание 4. Исследование TTL
Провести испытания при TTL 5, 30 и 120 секунд.
Определить, как срок жизни кэша влияет на количество обращений к основному источнику.
Сформулировать вывод о компромиссе между производительностью и актуальностью данных.

Задание 5. Анализ метрик
С использованием Grafana исследовать зависимости между:
- Hit Ratio;
- Source RPS;
- p95;
- интенсивностью HTTP-запросов.

На основании результатов определить, при каких условиях Redis обеспечивает наибольший выигрыш.

Проверка результатов
Проверить состояние Redis:
kubectl get pods -l app=redis

Проверить приложение:
kubectl get pods -l app=cache-app

Проверить доступность HTTP-endpoint:
curl "http://localhost:8080/data/1?cache=on"

Проверить наличие метрик:
curl "http://localhost:8080/metrics"

Проверить количество ключей Redis:
kubectl exec deployment/redis -- redis-cli DBSIZE

Контрольные вопросы

1. Что такое кэширование и зачем оно используется в высоконагруженных приложениях?
2. Какие задачи может решать Redis и чем внешний кэш отличается от локального кэша приложения?
3. Что представляет собой стратегия Cache-Aside?
4. Что означают Cache Hit и Cache Miss и как рассчитывается Hit Ratio?
5. Почему высокая доля попаданий не всегда гарантирует высокую производительность?
6. Что такое TTL и как изменение TTL влияет на нагрузку на источник и на актуальность данных?
7. Как кэширование влияет на p95 и p99 и почему при включении Redis максимальный RPS может не увеличиться?
8. Какие метрики позволяют оценить эффективность кэширования и как определить, насколько уменьшилось количество обращений к источнику?
9. Чем отличаются холодный и прогретый кэш, и почему результаты нагрузочного тестирования зависят от разнообразия запрашиваемых ключей?
10. Что такое Cache Stampede, почему Redis сам может стать узким местом, и в каких случаях кэширование нецелесообразно?